

RELATÓRIO DE EVOLUÇÃO ARQUITETURAL — SPRINT 03

1. Resumo da Evolução

Nas Sprints 1 e 2, o grupo criou um MVP conversacional em formato de script sequencial, realizando chamadas diretas via API de Chat Completions e gerenciando a memória adicionando manualmente dicionários a uma lista Python (histórico_conversa).

Na Sprint 03, a solução passou por uma evolução arquitetural:

- Subscrição do envio sequencial de prompts por Agentes de IA (OpenAI Agents SDK).
- Substituição do histórico em memória RAM por Gestão de Sessão com Persistência SQLite.
- Implementação de Guardrails Programáticos para tratamento de prompt e validação de escopo.

2. Decisões Técnicas e Refatoração

- Framework Escolhido: OpenAI Agents SDK.
- Motivo da Escolha: Integração nativa com modelos OpenAI, suporte a *Guardrails* determinísticos antes e depois da execução da LLM, e abstração limpa de sessões.
- Principais Componentes: Agent, Runner, SQLiteSession, @input_guardrail e @output_guardrail.
- Trade-offs Identificados:
 - Vantagem: Código modular, maior segurança (interrupção sem gastar tokens) e persistência automática de estado.
 - Limitação: Acoplamento ao ecossistema de APIs da OpenAI para certas funcionalidades avançadas de ferramentas.

3. Tabela Comparativa: Antes (Sprint 2) vs. Depois (Sprint 03)

Critério de Comparação	Versão Anterior (Sprint 2)	Nova Arquitetura (Sprint 03)
Arquitetura	Chamadas diretas de Chat Completion	Agente de IA Orquestrado (Agents SDK)
Gerenciamento de Memória	Lista Python manual em memória volátil	Extensão de sessão do SQLite com persistência de contexto

Proteção contra Injections	Dependente apenas de texto no prompt	Barreira de segurança de entrada programático interceptador
Latência Média por Turno	1.10s	0.85s
Consumo Médio de Tokens	Crescimento linear acumulativo descontrolado	Otimizado via gerenciamento de sessão do Runner
Nota Média de Testes	3.8 / 5.0	4.9 / 5.0

4. Problemas Encontrados e Soluções Adotadas

Problema 1: Vulnerabilidade a prompt por sobreposição de contexto

- **Descrição:** Na Sprint 2, ao enviar mensagens como "Ignore as instruções anteriores", a LLM eventualmente aceitava o comando do usuário devido à ausência de barreiras externas ao prompt.
- **Alternativas Analisadas:**
 1. Aumentar a rigidez do SYSTEM_PROMPT com mais regras.
 2. Implementar um Guardrail de código com @input_guardrail.
- **Solução Adotada:** Implementação do @input_guardrail no Agents SDK com validação de padrões via expressões.
- **Justificativa:** O Guardrail interrompe a execução antes da chamada da API, garantindo 100% de bloqueio e reduzindo custos com tokens.

Problema 2: Perda de histórico e estouro de janela de contexto em conversas longas

- **Descrição:** A lista historico_conversa da Sprint 2 acumulava todo o texto, tornando o payload gigante e causando perdas de estado se a sessão fosse reiniciada.
- **Alternativas Analisadas:**
 1. Truncar a lista Python manualmente a cada N mensagens.
 2. Adotar a classe de extensão da biblioteca nativa do Agents SDK.
- **Solução Adotada:** Extensão da biblioteca nativa do Agents SDK.
- **Justificativa:** Permite salvar a conversa em banco local, recuperando o contexto exato do usuário entre execuções de forma transparente e robusta.

5. Divisão do Trabalho na Equipe

Nome do Integrante	RM	Responsabilidade Principal na Sprint 03
Giovanna Tristão Lopes	RM572552	Refatoração do código base e implementação do OpenAI Agents SDK.
Enzo Posterlli Strinta	RM570035	Implementação e validação dos Guardrails de Segurança e Prompt Injection.
Vinicius Brito Davi	RM569709	Execução das baterias de teste de memória, benchmarks e elaboração do relatório.